The Grave-Shift Placement Problem

A Lexicographic Assignment Formulation

ShiftBuilder — Internal Maintenance, Gun Lake Casino

18 July 2026

Abstract

We give a formal specification of the nightly team-member placement problem solved by ShiftBuilder, and an algorithm for it. The operational requirement is a strict priority ordering over four objectives—coverage, rotation, preference, and skill—together with a family of hard constraints that are never tradeable. We show that this is precisely a *lexicographic assignment problem*: a maximum-cardinality bipartite matching for the dominant objective, refined by lexicographic vector comparison over the remainder. The formulation yields provable coverage optimality, exact infeasibility certificates via the deficiency form of Hall's theorem, and structural enforcement of the priority ordering. We contrast it with the scalarized (weighted-sum) formulation currently deployed, and identify the classes of defect that the lexicographic formulation renders unrepresentable.

1 Scope and motivation

Each night, a roster of approximately twenty-five team members must be assigned to approximately thirty-four board positions: ten zones, ten restroom stations, an administrative position, a smoking-room position, a flexible auxiliary row, and two bands of partial-shift overlap seats. The assignment is subject to hard physical and policy constraints, and is evaluated against four objectives whose relative priority has been ratified by the operator:

$$\mathsf{coverage} \succ \mathsf{rotation} \succ \mathsf{preference} \succ \mathsf{skill}. \tag{1}$$

The symbol $\succ$ in (1) denotes *priority*, not a weight ratio. The operator did not assert that one unit of coverage is worth some number of units of rotation; the assertion is that coverage decides, and the remaining objectives are consulted only among coverage-equal solutions. Section 4 makes this precise, and Remark 3 explains why encoding a priority ordering as a weighted sum is a lossy translation.

2 Notation

Definition 1 (Instance)**.** A placement instance is a tuple $\mathcal{I} = (T, S, \mathcal{R}, \mathrm{lock}, \mathrm{pin}, H, W, \theta)$ where

- T is the set of team members scheduled tonight, after removal of call-offs and dated unavailability;
- S is the ordered set of board positions, indexed by fill order;
- $\mathcal{R} \subseteq S$ is the set of *required* positions (restrooms and zones); we write $\mathcal{O} = S \setminus \mathcal{R}$ for the *optional* remainder;
- $\mathrm{lock} \subseteq S$ are operator-locked positions, each with an incumbent $\iota(s) \in T$;
- $\mathrm{pin} \subseteq S$ are preserved-but-unlocked existing placements, per the run's preserve policy;

- $H(t)$ is the placement history of t, a sequence of (*night*, *area*) events in decreasing night order;
- $W(t)$ is the set of placements already planned for t earlier in the current week solve;
- θ is the active engine configuration (weights and thresholds), and ζ a pseudorandom seed.

An *assignment* is a partial injection $\mu : S \rightharpoonup T$; $\mu(s) = t$ means t holds position s. Injectivity encodes the physical constraint that a person occupies at most one position per night. We write $\mathrm{dom}(\mu)$ for the filled positions and $\mathrm{cov}(\mu) = |\mathrm{dom}(\mu) \cap \mathcal{R}|$ for the coverage of μ.

2.1 Area identity

Rotation is not defined over positions but over *areas*. Let $\alpha : S \to \mathcal{A}$ be the area map. It is not injective: the two sides of a restroom pair share an area, because working either side of restroom 8 is operationally the same exposure, and overlap seats within a band are fungible. Zones and auxiliary roles are their own areas.

$$\alpha(\texttt{MRR8}) = \alpha(\texttt{WRR8}) = \texttt{RR8}, \qquad \alpha(\texttt{OL-PM-}k) = \texttt{OL-PM}, \qquad \alpha(\texttt{Z4}) = \texttt{Z4}. \tag{2}$$

2.2 Key normalization

Position identity is recorded in several vocabularies across the persistence layer: a user-interface form, a database form, a historical trail form, and a per-night ephemeral form for flexible auxiliary shells. Let Canon be the total function mapping any of these to canonical form.

Definition 2 (Boundary rule)**.** Every key entering the algorithm is normalized by Canon at the boundary. The algorithm is defined only over canonical keys. Canon is total over production key forms; an unrecognized key yields a distinguished value $\perp$, and CanPlace$(\cdot, \perp, \cdot)$ is false.

Definition 2 is stated as part of the algorithm rather than as an implementation note because the alternative—comparing keys drawn from distinct vocabularies—silently produces *false negatives* in every rotation predicate, and false negatives in a rotation predicate are indistinguishable from good news. A system in this state reports that a member has never worked a position they worked last week.

3 The feasibility gate

All hard constraints are collected into a single predicate. No other part of the algorithm may test eligibility.

Definition 3 (Gate)**.** CanPlace(t, s, r) holds when member t may occupy position s at relaxation level $r \in \{0, 1\}$. The constituent gates are:

Level 0 (never relaxed, at any r):

G_1 Position physics. Zone positions require a full-grave member; men's and women's restrooms require the corresponding gender; overlap seats require the matching partial-shift band; full-night positions exclude partial-shift members. Unknown gender and unknown position family both *fail closed*.

G_2 Locks. $s \in \mathrm{lock} \Rightarrow t = \iota(s)$.

G_3 Accommodations, from all recorded sources.

G_4 Schedule membership.

G_5 Operator rules, evaluated *within their declared scope*. A rule scoped to a position class constrains only that class.

G_6 Hard avoidance: declared position avoidance, and pairwise avoidance with an already-placed neighbour.

Level 1 (relaxed only when $r = 1$):

G_7 Rotation. The area $\alpha(s)$ must not occur among the three most recent events of $H(t)$ merged with $W(t)$. Continuity roles (the administrative position and auxiliary roles flagged as continuity) and overlap seats are exempt by policy.

The ladder has exactly two rungs. There is no level at which a Level-0 gate yields. Every placement made at $r = 1$ carries a relaxation record in its provenance, and Section 9 shows how the *number* of such records is itself minimized.

Remark 1. The placement of G_6 at Level 0 is a policy choice, and it is the choice the written operator contract makes: a hard avoidance is described there as a constraint, not a cost. Should the operator wish avoidance to be tradeable against coverage, the correct edit is to demote G_6 to a term in π (Section 4), not to introduce a third relaxation rung.

4 The objective

Definition 4 (Score)**.** For a candidate placement of t at s under partial assignment μ,

$$\textsc{Score}(t, s, \mu) \;=\; \big(\rho(t, s),\ \pi(t, s, \mu),\ \kappa(t, s)\big) \in \mathbb{Z} \times \mathbb{R} \times \mathbb{R}, \tag{3}$$

with components defined as follows.

Rotation. Let $\sigma_{30}(t, a)$ be the number of the last thirty worked nights in which t occupied area a, let $\gamma(t, a)$ be the number of days since t last occupied a, and let sf and l_5 be indicators for a same-side restroom-family repeat within the critical window and a same-area repeat within the last five events. Then

$$\rho(t, s) \;=\; 100 \;-\; 55\,\mathbf{1}[\mathrm{sf}] \;-\; 25\min\big(\sigma_{30}(t, \alpha(s)),\, 3\big) \;-\; 10\,\mathbf{1}[\mathrm{l}_5] \;+\; \min(7, \lfloor \gamma(t, \alpha(s))/8 \rfloor)\,. \tag{4}$$

The prior-three same-area condition does *not* appear in (4); it is the gate G_7, and a gate is not a term.

Preference. With $h, f \in \mathbb{Z}$ the summed hard and soft preference signals for (t, s) and $a(t, s, \mu)$ the pair-affinity signal against already-placed neighbours,

$$\pi(t, s, \mu) \;=\; \mathrm{clamp}(h, 0, 1) \;+\; 0.4\,\mathrm{clamp}(f, -1, 1) \;+\; 0.5\,a(t, s, \mu). \tag{5}$$

Every term is clamped, so π is bounded by construction. Hard *avoidance* does not appear here; it is G_6.

Skill. With $\varsigma(t)$ the skill score and $d(s)$ the position difficulty,

$$\kappa(t, s) \;=\; \max\big(-1,\ -|\varsigma(t) - d(s)|\big), \qquad \kappa(t, s) \geq -0.4 \ \text{ whenever } \varsigma(t) \geq d(s). \tag{6}$$

The floor encodes that being overqualified is not a defect.

Definition 5 (Lexicographic order)**.** For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, $\mathbf{u} \succ_{\mathrm{lex}} \mathbf{v}$ if and only if there exists k with $u_k > v_k$ and $u_i = v_i$ for all $i < k$.

Candidates are compared by $\succ_{\mathrm{lex}}$ on Score, with ties broken first by the member's tie-break rank and then by identifier. Since identifiers are unique, the comparison is total and the algorithm therefore deterministic.

Remark 2 (Integrality of ρ). The rotation component is integer-valued by construction. This is deliberate. Under a strict lexicographic order, a real-valued dominant component makes exact ties a measure-zero event, and the subordinate components are never consulted: two boards differing by 10^{-4} in rotation would be treated as rotation-distinct, and preference would never speak. Quantization creates a band of genuine rotation-equivalence within which the lower objectives decide. The granularity of that band is the one tuning decision this formulation genuinely requires.

Remark 3 (Why not a weighted sum). A scalarization $\Phi = w_1 f_1 + w_2 f_2 + w_3 f_3$ reproduces lexicographic behaviour only if each weight dominates the entire achievable range of everything beneath it,

$$w_i \;>\; \sum_{j>i} w_j \cdot \big(\max f_j - \min f_j\big). \tag{7}$$

Condition (7) depends on the *ranges* of the subordinate objectives. It is therefore invalidated by any retuning that widens a lower objective, by any unclamped sum, and by the accumulation of several simultaneously matching preference rows. The failure is silent: no constraint is violated and no exception is raised; the priority ordering simply inverts. Lexicographic comparison has no analogue of (7) to violate, because no arithmetic is performed across objectives.

5 The night algorithm

Let $G_r = (T \cup S, E_r)$ be the bipartite *eligibility graph* at relaxation level r, with $\{t, s\} \in E_r$ exactly when $\textsc{CanPlace}(t, s, r)$. Coverage is a property of matchings in G_r; the remaining objectives select among them.

Two structural points distinguish Algorithm 1 from a greedy fill.

First, Phase B decides *which* positions can be filled before Phase C decides *who* fills them. A greedy walk conflates these, and in doing so commits early choices that can strand a later position: if the only member eligible for a restroom has already been consumed by a zone, a greedy walk has no move that recovers. The augmenting path of Phase B is exactly that move—it relocates an already-placed member through a chain of reassignments—and it is found automatically rather than written as a special case.

Second, Phase C's oracle $\textsc{Feasible}(\mu, s \mapsto t)$ tests whether fixing t at s still admits a maximum matching on the remainder. Without it, an optimal-looking local choice could destroy the coverage guarantee established in Phase B.

6 Correctness

Theorem 1 (Deficiency certificate)**.** *Let G_0 be the eligibility graph and $\nu(G_0)$ the size of a maximum matching. Then*

$$|\mathcal{R}| - \nu(G_0) \;=\; \max_{A \subseteq \mathcal{R}} \big(|A| - |N(A)|\big), \tag{8}$$

where $N(A) \subseteq T$ is the neighbourhood of A. Any maximizing A is a witness that $|A| - |N(A)|$ positions of A must go unfilled.

Proof. This is the deficiency form of Hall's theorem, equivalent to Kőnig's theorem on bipartite graphs. The maximizing set A is recoverable from the alternating-path structure produced by the matching algorithm: take the unmatched required positions together with all positions reachable from them by alternating paths. □

Theorem 1 is the algorithm's explanation facility, not merely its analysis. When five women's restrooms draw on only three eligible members, the returned set A names those five positions and

Algorithm 1 NightSolve($\mathcal{I}, \zeta$)

Input: instance $\mathcal{I}$, seed ζ
Output: assignment μ, provenance, scorecard, certificates

Phase A — feasibility census
$E_0 \leftarrow \{\, \{t, s\} : s \in \mathcal{R},\ \textsc{CanPlace}(t, s, 0) \,\}$
$M^* \leftarrow \textsc{MaxMatching}(G_0)$ ▷ Hopcroft–Karp
for all $s \in \mathcal{R}$ unmatched in M^* **do**
    $\mathrm{cert}(s) \leftarrow \textsc{DeficientSet}(G_0, s)$ ▷ Theorem 1
end for
Phase B — coverage
$\mu \leftarrow \{\, s \mapsto \iota(s) : s \in \mathrm{lock} \,\} \cup \{\, s \mapsto \mu_0(s) : s \in \mathrm{pin} \,\}$
$\textsc{Augment}(\mu,\ E_0$ restricted to unassigned members$)$
if some $s \in \mathcal{R}$ remains open **then**
    $\textsc{Augment}(\mu,\ E_0)$ along shortest augmenting paths ▷ may displace pinned members
    record each displacement as moved-for-coverage
end if
if some $s \in \mathcal{R}$ remains open **then**
    $E_1 \leftarrow E_0 \cup \{\, \{t, s\} : \textsc{CanPlace}(t, s, 1) \,\}$
    $\textsc{Augment}(\mu,\ E_1)$ for open required positions only
    tag each $E_1 \setminus E_0$ edge used with relaxation$= G_7$
end if
Phase C — selection
for all $s \in \mathcal{R}$ in fill order, $s \notin$ lock **do**
    $C \leftarrow \{\, t$ unassigned : $\textsc{CanPlace}(t, s, r(s)) \wedge \textsc{Feasible}(\mu, s \mapsto t) \,\}$
    $t^* \leftarrow \mathrm{lex\,max}_{t \in C}\, \textsc{Score}(t, s, \mu)$
    $\mu(s) \leftarrow t^*$; append s to $W(t^*)$
end for
for all $s \in \mathcal{O}$ in fill order **do**
    as above, with $r = 0$ and no displacement permitted
end for
Phase D — refinement
for $k = 1$ **to** K (in seeded order) **do**
    propose a swap or relocation μ' with both endpoints admissible at $r = 0$, locks untouched, $\mathrm{cov}(\mu') = \mathrm{cov}(\mu)$
    if $\Sigma\textsc{Score}(\mu') \succ_{\mathrm{lex}} \Sigma\textsc{Score}(\mu)$ **then**
        $\mu \leftarrow \mu'$
    end if
end for
Phase E — advisory overlay (optional)
obtain proposed overrides from the language model, which is given the rules, the per-position candidate rankings, and tools calling the *same* CanPlace and Score
accept an override only if $\textsc{CanPlace}(\cdot, \cdot, 0)$ holds, cov is unchanged, and $\Sigma\textsc{Score}$ does not regress
Phase F — verification
repeat
    $V \leftarrow$ violations of Invariants **I1**–**I5**
    revert positions implicated in V to their Phase-C values
until $V = \emptyset$
return μ, provenance, scorecard (with V), certificates

the deficiency is two. This is a proof of impossibility localized to the responsible positions, and it subsumes special-cased headcount arithmetic: a gender shortfall is not a separate calculation but an instance of deficiency.

Proposition 1 (Coverage optimality)**.** *Let r^* be the highest relaxation level used by Phase B. Then* $\mathrm{cov}(\mu)$ *is maximum over all assignments satisfying the gates at levels $\leq r^*$ and respecting* lock*.*

Proof. Locked positions contribute forced edges; contract them, which reduces the problem to matching on the residual graph. By Berge's lemma a matching is maximum if and only if it admits no augmenting path. Phase B searches for augmenting paths over the full residual graph—first among unassigned members, then, if positions remain open, over paths that displace pinned members—and terminates only when none exists. Hence the matching is maximum at level 0. If required positions remain open, the edge set is extended to E_1 and the search repeated, yielding a maximum matching at level 1. Maximality at r^* follows. □

Proposition 2 (Selection preserves coverage)**.** *If Phase C admits only candidates passing* Feasible*, then on termination* $\mathrm{cov}(\mu)$ *equals the value established in Proposition 1.*

Proof. By induction on the fill order. The hypothesis is that after k selections the partial assignment extends to a maximum matching. For $k = 0$ this is Proposition 1. For the inductive step, Feasible admits t at s exactly when $\mu \cup \{s \mapsto t\}$ extends to a maximum matching; such a t exists because the maximum matching guaranteed by the hypothesis assigns *some* member to s, and that member passes the test. Selecting any admitted candidate preserves the hypothesis. □

Proposition 3 (Priority soundness)**.** *Let μ, μ' be assignments with* $\mathrm{cov}(\mu) > \mathrm{cov}(\mu')$*. Then μ is preferred, irrespective of ρ, π, κ. Analogously, if coverage is equal and $\Sigma\rho(\mu) > \Sigma\rho(\mu')$, then μ is preferred irrespective of π, κ.*

Proof. Immediate from the definition of $\succ_{\mathrm{lex}}$: comparison is decided at the first index at which the vectors differ, and no subsequent component is examined. □

Proposition 3 is trivial as mathematics and load-bearing as engineering: it is exactly the guarantee that condition (7) fails to provide robustly. Under lexicographic comparison the ratified ordering (1) holds by the type of the comparison, and is therefore invariant under retuning of any component's internal weights.

Proposition 4 (Determinism)**.** *NightSolve and WeekSolve are pure functions of $(\mathcal{I}, \zeta)$.*

Proof. Candidate comparison is a total order (lexicographic on Score, then on the tie-break pair, whose second element is unique). All iteration orders derive from ζ by a fixed generator. No component reads ambient state: in particular the rotation component is computed against an explicit *before-night* cursor rather than the wall clock. □

7 The week algorithm

The week solve is the night solve folded over the week, with a shared rolling history, followed by a cross-night polish. It is not a second algorithm; in particular it invokes the same gate, so no constraint enforced on a single night can be evaded across a week.
The week score governing the polish loop of Algorithm 2 is

$$\textsc{WeekScore} = \Big(\textstyle\sum_n \mathrm{cov}(\mu_n),\ -\sum_n \mathrm{rep}(\mu_n),\ \sum_n \Sigma\rho,\ \sum_n \Sigma\pi,\ \sum_n \Sigma\kappa\Big), \tag{9}$$

Algorithm 2 WEEKSOLVE$(N, \mathcal{I}, \zeta)$ for nights $N = (n_1, \ldots, n_7)$

$W \leftarrow \emptyset$
for all $n \in N$ in chronological order **do**
    $\mathcal{I}_n \leftarrow$ instance for n with history merge(H, W)
    $\mu_n \leftarrow$ NIGHTSOLVE$(\mathcal{I}_n, \zeta \oplus n)$
    $W \leftarrow W \cup \mu_n$ ▷ planned nights are visible to later nights
end for
for $k = 1$ **to** K_W (in seeded order) **do**
    propose exchanging placements across nights $n_i \neq n_j$, both endpoints admissible at $r = 0$, coverage of both preserved
    if WEEKSCORE improves lexicographically **then**
        accept, and **rebuild** the context of every night after $\min(n_i, n_j)$
    end if
end for
return $(\mu_n)_{n \in N}$, week scorecard, relaxation ledger

with rep counting within-week area repeats. The context rebuild that accompanies each accepted exchange is required for soundness: an exchange alters the history that later nights were evaluated against, and evaluating a night against a board that no longer exists admits exchanges whose true downstream effect is negative.

8 The placement matrix as a derived view

The placement matrix records, per member and per zone, exposure counts over trailing calendar windows. We observe that it is a pure function of history and the clock:

$$\mathcal{M}(t, a) \;=\; \big(c_{28}(t, a),\; c_{56}(t, a),\; \ell(t, a)\big), \qquad c_w(t, a) \;=\; \big|\{\, e \in H(t) : \alpha(e) = a,\; \delta(e) \le w \,\}\big|, \tag{10}$$

where $\delta(e)$ is the age of event e in days and $\ell(t, a)$ the most recent night on which t occupied a. Consequently

$$\mathcal{M} \;\equiv\; f(H, \mathrm{now}), \tag{11}$$

and materializing $\mathcal{M}$ is a caching decision rather than a design one. At the operative scale—order 10^3 history rows—evaluating (10) on read is submillisecond, and the materialized form should be regarded as an optimization to be justified, not a default.

The distinction matters because a materialized $\mathcal{M}$ admits states that (10) cannot express. A stored count can disagree with history, either because a mutation path failed to trigger a rebuild, or because trailing windows were computed at a past instant and no longer correspond to the present one. Neither failure is representable when $\mathcal{M}$ is evaluated on read. If materialization is retained, the governing invariant is that $\mathcal{M}$ be reconstructible from H alone, identically, at any time; a rebuild triggered by every mutation that touches history, together with a periodic sweep, then bounds staleness while eliminating incorrectness.

9 An equivalent integer program

The same problem admits a compact integer formulation, useful as an independent oracle against which the staged algorithm can be validated.

$$
\begin{aligned}
&\text{variables} && x_{ts} \in \{0,1\} && t \text{ occupies } s && (12)\\
&&& y_s \in \{0,1\} && s \in \mathcal{R} \text{ left open} && (13)\\
&&& z_{ts} \in \{0,1\} && \text{edge used under relaxation } G_7 && (14)\\
&\text{subject to} && \textstyle\sum_s x_{ts} \le 1 && \forall t \in T && (15)\\
&&& \textstyle\sum_t x_{ts} + y_s = 1 && \forall s \in \mathcal{R} && (16)\\
&&& \textstyle\sum_t x_{ts} \le 1 && \forall s \in \mathcal{O} && (17)\\
&&& x_{ts} = 0 && \forall (t,s) \text{ failing a Level-0 gate} && (18)\\
&&& x_{ts} \le z_{ts} && \forall (t,s) \text{ failing only } G_7 && (19)\\
&&& x_{\iota(s)s} = 1 && \forall s \in \text{lock} && (20)
\end{aligned}
$$

The objective is optimized in stages, each stage constraining the next:

1. minimize $\sum_{s\in\mathcal{R}_1} y_s$, then $\sum_{s\in\mathcal{R}_2} y_s$ (coverage, by tier)
2. minimize $\sum_{t,s} z_{ts}$ (fewest relaxations)
3. maximize $\sum_{t,s} \rho(t,s)\, x_{ts}$ (rotation)
4. maximize $\sum_{t,s} \pi(t,s)\, x_{ts}$ (preference)
5. maximize $\sum_{t,s} \kappa(t,s)\, x_{ts}$ (skill)

Staged optimization—each stage fixing the optimum of its predecessor as a constraint—realizes (1) without weight arithmetic, and is the standard formulation of preemptive goal programming. Since π depends on the assignment through pair affinity, that term is either linearized with auxiliary products or, in practice, evaluated in the final stage over a restricted candidate set.

Remark 4. At $|T| \approx 25$ and $|S| \approx 34$ the program has fewer than a thousand binaries and solves in milliseconds. Running it as a shadow validator alongside the staged algorithm yields a disagreement certificate—a concrete board on which the two differ—which is a considerably stronger signal than an assertion failure.

10 Invariants

I1 μ is injective: no member holds two positions.

I2 Locked positions are identical across any run.

I3 No output placement violates a Level-0 gate.

I4 $\mathrm{cov}(\mu)$ attains the matching bound of Proposition 1: coverage is optimal, not best-effort.

I5 Every G_7 waiver appears in the relaxation ledger, and the number of waivers is minimized.

I6 NightSolve and WeekSolve are pure in $(\mathcal{I}, \zeta)$.

I7 $\mathcal{M} \equiv f(H, \mathrm{now})$, over a single history store with singular (*night*, *position*) ownership closed under every mutation path.

I8 Canon is total over production key forms; unrecognized keys fail closed.

I9 Explanations—score vectors, deficiency certificates, relaxation ledgers—are byproducts of solving, not reconstructions after it.

Invariants **I1**–**I5** are checked in Phase F and re-checked after any repair, so a result reporting no violations has been shown to have none. **I3** and **I8** are amenable to property-based testing over randomized rosters, configurations, and seeds.

11 Complexity

Let $n = |T|$, $m = |S|$, $E = |E_0| \leq nm$.

Phase	Operation	Complexity	At n=25, m=34
A	Hopcroft–Karp	$O(E\sqrt{n+m})$	$\sim 10^4$ ops
A	Deficiency certificates	$O(E)$ per open position	negligible
B	Augmentation	$O(E\sqrt{n+m})$	$\sim 10^4$ ops
C	Selection with oracle	$O(m \cdot n \cdot E)$ worst case	$\sim 10^6$ ops
D	Refinement	$O(K)$, budget-bounded	tunable
F	Verification	$O(m)$ per pass	negligible

The whole solve completes well within ten milliseconds. No approximation is required at this scale: provable coverage optimality and full determinism are available at no practical cost, and any heuristic in the implementation should be justified by explanatory or incremental-update requirements rather than by performance.

12 Relation to the deployed implementation

The deployed system is staged in the same shape—plan, optimize, advise, verify—and that shape is worth preserving, since it is the source of the provenance that makes engine output auditable by the operator. The formulation above substitutes for its internals as follows.

Deployed	**Specified here**	**Consequence**
Greedy fill with rescue and backfill passes	Maximum matching, then selection under a feasibility oracle	Coverage provably optimal; stranding unrepresentable (Prop. 1, 2)
Scalar weighted sum with hard gates	Lexicographic vector (ρ, π, κ)	Ordering (1) holds by construction (Prop. 3, Rem. 3)
Three distinct relaxation behaviours	One two-rung ladder inside CanPlace	Single semantics; every waiver ledgered
Multiple history stores and key vocabularies	One store, Canon at the boundary	Rotation false-negatives eliminated as a class (Def. 2)
Materialized matrix with refresh choreography	Derived view; materialization optional	Staleness bounded, incorrectness unrepresentable (§8)
Repair without re-verification	Verify–repair–re-verify to fixpoint	A clean report is a proof, not an assumption
Feasibility by headcount arithmetic	Deficiency certificates	Exact, per-position, gender split subsumed (Thm. 1)

Two companion documents complete the set:

`PLACEMENT_ARCHITECTURE.md` — a narrative description of the system as built, including the persistence layer and the explanation surfaces.

`PLACEMENT_REVIEW_2026-07-18.md` — a catalogue of the specific divergences between the deployed system and this specification, with severity ranking and a suggested remediation order.